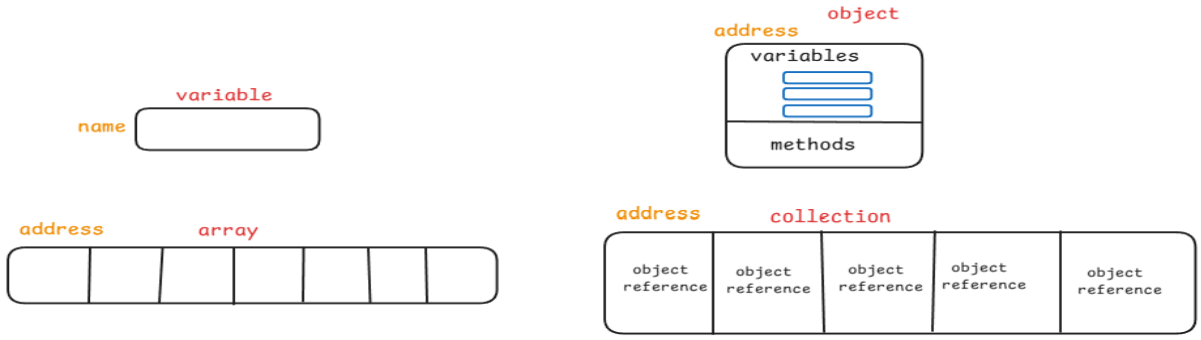
COLLECTION FRAMEWORK

COLLECTION:

Collections in Java are used to store, retrieve, manipulate, and communicate aggregate data. Typically, they represent data items that form a natural group, such as a telephone directory (a mapping of names to phone numbers).

Collection is a collection of objects or a group of objects treated as a single unit. The pictorial representation of Collection compared with other ways to store data, looks like this:



Advantages of Collection over Arrays:

When we already have Arrays to store multiple data in contiguous memory blocks, why to go for Collection? Following are the drawbacks of Arrays and corresponding advantages of Collection.

```
DRAWBACKS OF ARRAY:
1. size: fixed
2. indexing: mandatory
3. homogenous elements
4. less in-built methods
5. no data structures
```

```
ADVANTAGES OF COLLLECTION:
1. size is not fixed, flexible
2. indexing is optional
3. homogenous and heterogenous elements
4. numerous in-built methods to perform CRUD operations
5. data structures like List, Queue, Set, Map
```

COLLECTION FRAMEWORK:

Collection framework is a group of

- ❖ in-built **interfaces** and
- ❖ their implementing **concrete classes**

which define a mechanism to create different types of collection objects that follow different data structures such as List, Queue, Set, Stack, Map or dictionaries.

Implementing Collection framework:

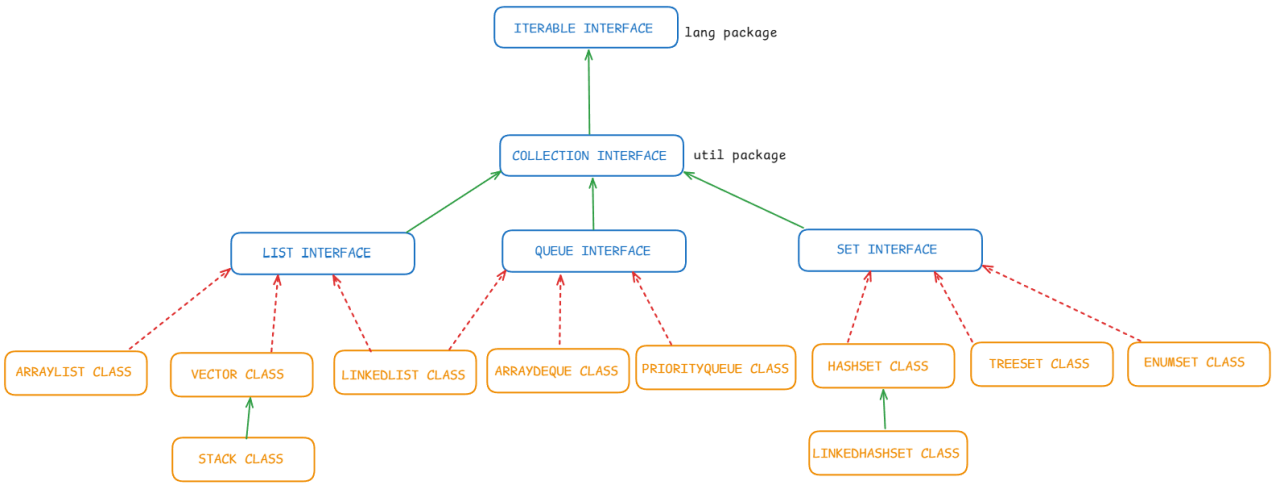
In order to implement collection framework, we have to make use of one of the hierarchies out of,

- ➢ **Collection hierarchy**
- ➢ **Map hierarchy**

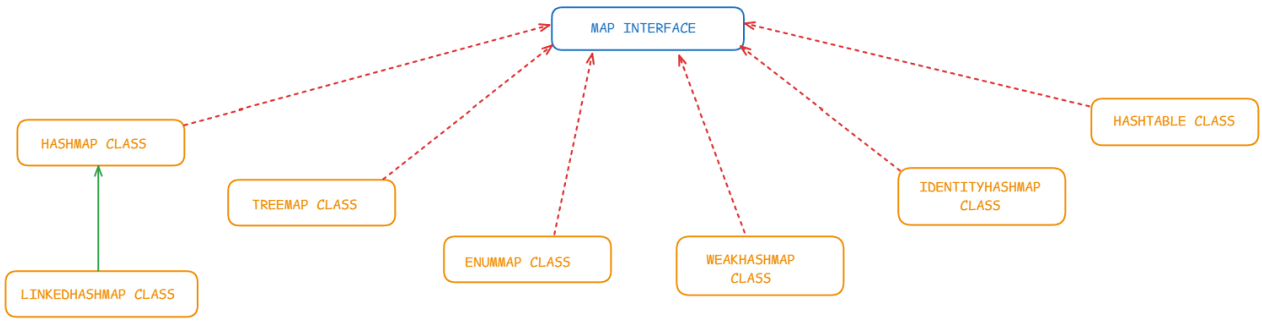
They are interfaces which have implementing concrete classes. As objects cannot be created to interfaces, we should create objects to concrete child classes.

PICTORIAL REPRESENTATION OF COLLECTION FRAMEWORK:

Collection Hierarchy:



Map Hierarchy:



Package of Collection Framework: util package, except Iterable interface

CHARACTERISTICS OF LIST INTERFACE:
1. it is in-built interface
2. it is present in java.util package
3. insertion order is maintained
4. indexing is present
5. duplicates are allowed
6. multiple null are allowed

CHARACTERISTICS OF ARRAYLIST CLASS:
1. it is a concrete implementing class
2. it is present in java.util package
3. insertion order is maintained
4. indexing is present
5. duplicates are allowed
6. multiple null are allowed
7. constructors:
ArrayList()
ArrayList(Collection c)
8. Storage of data:
stored in the form of dynamic array

CHARACTERISTICS OF LINKEDLIST CLASS:
1. it is a concrete implementing class
2. it is present in java.util package
3. insertion order is maintained
4. indexing is present
5. duplicates are allowed
6. multiple null are allowed
7. constructors:
LinkedList()
LinkedList(Collection c)
8. Storage of data:
stored in the form of nodes

CRUD operations: Create, Read, Update, Delete

As we have seen that Collection has numerous methods to perform operations on stored data (elements stored as objects) such as,

1. Add elements
2. Access elements
3. Search elements
4. Remove elements
5. Sort elements

IN-BUILT METHODS TO PERFORM THESE OPERATIONS:

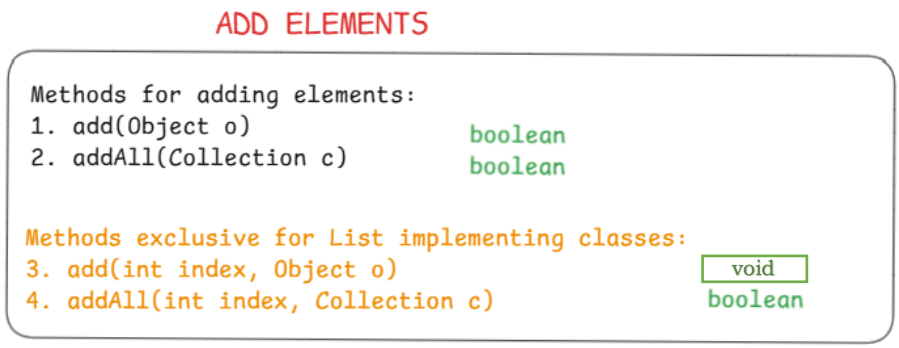
1. **ADD ELEMENTS**:

When elements are added, the values of primitive datatypes are auto-boxed (converted into Wrapper class type object) and implicitly upcasted (from Wrapper class type to Object class type) and hence stored in the Collection as objects.

```
HOW ELEMENTS ARE CONVERTED TO OBJECTS?
1. auto-boxing
2. implicit upcasting

int value
to
Integer type object (for generic)
to
Object type object (for non-generic)
```

In-built methods:



Generic and Non-generic Collection:

By specifying the datatype of elements to be added in conical braces, a collection, here ArrayList can be made as generic ArrayList. Then only homogenous elements can be added.

```
3  import java.util.ArrayList;
4
5  public class Demo {
6      public static void main(String[] args) {
7      ArrayList a = new ArrayList(); // NON GENERIC ARRAYLIST // HETEROGENOUS ELEMENTS
8      ArrayList<Integer> b = new ArrayList<Integer>(); // GENERIC ARRAYLIST // HOMOGENOUS ELEMENTS
9      }
10 }
11
```

```java
package training;

import java.util.ArrayList;
import java.util.Arrays;

class AddElements
{
    public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<Integer>();
        //ArrayList() constructor calling statement
        a.add(11);
        a.add(22);
        a.add(33);
        a.add(44);
        System.out.println(a);
        ArrayList<Integer> b = new ArrayList<Integer>(Arrays.asList(10,20,30));
        //ArrayList(Collection c) constructor calling statement
        System.out.println(b);
        a.addAll(b);
        System.out.println(a);
        b.addAll(1, a);
        System.out.println(b);
    }
}

```

```
<terminated> AddElements [Java Application] C:\Users\na
[11, 22, 33, 44]
[10, 20, 30]
[11, 22, 33, 44, 10, 20, 30]
[10, 11, 22, 33, 44, 10, 20, 30, 20, 30]
```

2. ACCESS ELEMENTS:

There are five ways to access elements present in a Collection. The elements or objects of the Collection are fetched and printed on the console one by one. This operation is called accessing or fetching all elements by iteration or traversal.

ACCESS ELEMENTS

FETCH/ACCESS/TRAVERSE/ITERATE

WAYS TO ACCESS ELEMENTS:
1. for each loop
2. iterator method

WAYS EXCLUSIVE FOR LIST IMPLEMENTING CLASSES
3. listIterator method
4. listIterator(int index) method
5. get(int index) method

Direction of Traversal:

- We can fetch the elements of a Collection both in forward (left to right) and reverse (right to left) direction.
- Also, when the index of element is passed, the fetching can be started in a desired position and not essentially from start or end of the Collection.

FORWARD TRAVERSAL OF ELEMENTS:
1. for each loop
2. iterator method
3. listIterator()

FORWARD AND REVERSE TRAVERSAL OF ELEMENTS:
4. listIterator(int index)
5. get(int index) using for loop

FOR EACH LOOP:

For Each Loop is also called advanced or enhanced for loop. This for each loop can be used to iterate elements of an array or a Collection.

```
SYNTAX OF FOR EACH LOOP

for(data_type variable_name : array/collection)
{
    //statements;
}
```

CHARACTERISTICS OF FOR EACH LOOP:

1. Datatype of the Array or Collection to be passed is mentioned.
2. Any variable name can be used and that name is used in iteration.
3. Name of array or collection whose elements are to be iterated has to be passed after a colon.
4. This loop does the iteration by itself.
5. It can iterate elements only in the forward direction.

Example programs:

We have taken two examples to demonstrate accessing elements:

1. Non-generic ArrayList with different primitive type elements
2. Generic ArrayList with custom datatype, that is ClassName type

to explain all the five ways of accessing the elements.

Non-generic ArrayList with primitive type elements:

```
1  package training;
2
3  import java.util.ArrayList;
6
7  class Example2 {
8      public static void main(String[] args) {
9          ArrayList a = new ArrayList();
10         // a is object reference of ArrayList type
11         a.add(12);
12         a.add('g');
13         a.add(65.47);
14         a.add(true);
15         System.out.println(a);
16         //fetch or access elements in the list by iteration
17         System.out.println("***************FOR EACH LOOP***************");
18         for(Object ele : a) {
19             System.out.println(ele);
20         }
21         System.out.println("**************ITERATOR METHOD***********");
22         Iterator i = a.iterator();
23         // i is object reference of Iterator type
24         while(i.hasNext()) {
25             System.out.println(i.next());
26         }
27         System.out.println("****************LIST ITERATOR METHOD**************");
28         ListIterator li = a.listIterator();
29         // li is object reference of ListIterator type
30         while(li.hasNext()) {
31             System.out.println(li.next());
32         }
33         System.out.println("****************LIST ITERATOR (INT INDEX) METHOD**************");
34         ListIterator re = a.listIterator(a.size()-1);
35         // re is object reference of ListIterator type
36         while(re.hasPrevious()) {
37             System.out.println(re.previous());
38         }
39         System.out.println("******************GET(INT INDEX) METHOD******************");
40         for(int x = a.size()-2; x>=0; x--) {
41             System.out.println(a.get(x));
42         }
43     }
44 }
45
```

```
<terminated> Example2 [Java Application] C:\Users\naray\.p2\pool\plugins\org.eclipse.ju
[12, g, 65.47, true]
***************FOR EACH LOOP***************
12
g
65.47
true
**************ITERATOR METHOD***********
12
g
65.47
true
****************LIST ITERATOR METHOD**************
12
g
65.47
true
****************LIST ITERATOR (INT INDEX) METHOD**************
65.47
g
12
******************GET(INT INDEX) METHOD******************
65.47
g
12
```

Generic ArrayList with non-primitive custom datatype (ClassName) elements:

```
class Example3 {
    public static void main(String[] args) {
        ArrayList<Employee> a = new ArrayList<Employee>();
        a.add(new Employee(101, "Popeye"));
        a.add(new Employee(102, "Bluto"));
        a.add(new Employee(103, "Spike"));
        System.out.println(a);
        System.out.println("************FOR EACH LOOP************");
        for(Employee ele : a) {
            System.out.println(ele);
        }
        System.out.println("********ITERATOR METHOD**********");
        Iterator<Employee> i = a.iterator();
        while(i.hasNext()) {
            System.out.println(i.next());
        }
        System.out.println("***********LIST ITERATOR METHOD**********");
        ListIterator<Employee> li = a.listIterator();
        while(li.hasNext()) {
            System.out.println(li.next());
        }
        System.out.println("************LIST ITERATOR(INT INDEX) METHOD***********");
        ListIterator<Employee> re = a.listIterator(a.size()-1);
        while(re.hasPrevious()) {
            System.out.println(re.previous());
        }
        System.out.println("**********GET(INT INDEX) METHOD************");
        System.out.println("single element: " + a.get(2));
        System.out.println("FORWARD TRAVERSAL");
        for(int x=2; x<a.size(); x++) {
            System.out.println(a.get(x));
        }
        System.out.println("REVERSE TRAVERSAL");
        for(int y=a.size()-1; y>0; y--) {
            System.out.println(a.get(y));
        }
    }
}
```

```
package training;

import java.util.ArrayList;

class Employee{
    int eid;
    String name;
    Employee(int eid, String name){
        this.eid = eid;
        this.name = name;
    }
    public String toString() {
        return "ID: " + eid + " NAME: " + name;
    }
}
```

Declaration | Console ×

<terminated> Example3 [Java Application] C:\Users\naray\.p2\pool\plugins\org.eclipse.justj.openjd

```
[ID: 101 NAME: Popeye, ID: 102 NAME: Bluto, ID: 103 NAME: Spike]
************FOR EACH LOOP************
ID: 101 NAME: Popeye
ID: 102 NAME: Bluto
ID: 103 NAME: Spike
********ITERATOR METHOD**********
ID: 101 NAME: Popeye
ID: 102 NAME: Bluto
ID: 103 NAME: Spike
***********LIST ITERATOR METHOD**********
ID: 101 NAME: Popeye
ID: 102 NAME: Bluto
ID: 103 NAME: Spike
************LIST ITERATOR(INT INDEX) METHOD***********
ID: 102 NAME: Bluto
ID: 101 NAME: Popeye
**********GET(INT INDEX) METHOD************
single element: ID: 103 NAME: Spike
FORWARD TRAVERSAL
ID: 103 NAME: Spike
REVERSE TRAVERSAL
ID: 103 NAME: Spike
ID: 102 NAME: Bluto
```

3. **SEARCH ELEMENTS**:
   - ❖ There are two methods, one to search elements and another to search a Collection that return a boolean value.
   - ❖ Two other methods to get the index position and last index position of the element or object that is passed.

SEARCH ELEMENTS

contains(Object o)	boolean
containsAll(Collection c)	boolean
indexOf(Object o)	int
lastIndexOf(Object o)	int

```
1  package training;
2
3  import java.util.ArrayList;
5
6  class SearchElements
7  {
8      public static void main(String[] args) {
9          ArrayList<Integer> a = new ArrayList<Integer>(Arrays.asList(3,12,47,12,4,12,41));
10             System.out.println(a);
11             System.out.println(a.contains(47));
12             System.out.println(a.contains(135));
13             System.out.println(a.indexOf(12));
14             System.out.println(a.indexOf(99));
15             System.out.println(a.lastIndexOf(12));
16             System.out.println(a.lastIndexOf(38));
17             ArrayList<Integer> b = new ArrayList<Integer>(Arrays.asList(12));
18             System.out.println(b);
19             boolean res = a.containsAll(b);
20             System.out.println(res);
21         }
22 }
23
```

```
<terminated> SearchElements [Java Applic
[3, 12, 47, 12, 4, 12, 41]
true
false
1
-1
5
-1
[12]
true
```

4. **REMOVE ELEMENTS**:
   - ❖ There are five methods to remove elements from a Collection.
   - ❖ For int and char type values, to avoid ambiguity, only remove(int index) method works.
   - ❖ removeAll(Collection c), retainAll(Collection c) and clear() are used to remove multiple elements at a time.

REMOVE ELEMENTS

remove(Object o) (except int and char)	boolean
remove(int index)	object
removeAll(Collection c)	boolean
retainAll(Collection c)	boolean
clear()	void

```java
package training;

import java.util.ArrayList;

class RemoveElements {
    public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<Integer>(Arrays.asList(12,45,6,74,15));
        System.out.println(a);
        System.out.println(a.remove(1));
        //a.remove(74); // index out of bounds exception
        ArrayList b = new ArrayList(Arrays.asList('f', 14, 45.68, true, "hello"));
        System.out.println(b);
        //b.remove('f'); // index 102 out of bounds (ASCII for f = 102)
        b.remove(0);
        b.remove("hello");
        b.remove(45.68);
        System.out.println(b);
        ArrayList<Integer> c = new ArrayList<Integer>(Arrays.asList(12,45));
        System.out.println(c);
        a.removeAll(c);
        System.out.println(a);
        a.addAll(c);
        System.out.println(a);
        a.retainAll(c);
        System.out.println(a);
        a.clear();
        System.out.println(a);
        System.out.println(a.size());
        System.out.println(a.isEmpty());
        System.out.println(c.isEmpty());
    }
}
```

```
<terminated> Example4 [Java Application]
[12, 45, 6, 74, 15]
45
[f, 14, 45.68, true, hello]
[14, true]
[12, 45]
[6, 74, 15]
[6, 74, 15, 12, 45]
[12, 45]
[]
0
true
false
```

5. **SORT ELEMENTS**:

- For sorting elements in a Collection, we have to make use of static methods of in-built class called **Collections class**, present in lang package.

SORT ELEMENTS

```
Collections.sort(List)        ascending order
Collections.reverse(List)     reverse, and not descending order
```

Collection is an interface in util package

Collections is a class in util package

```java
package training;

import java.util.ArrayList;

class SortElements {
    public static void main(String[] args) {
    ArrayList<Integer> a = new ArrayList<Integer>(Arrays.asList(14,3,14,2,68,12));
        System.out.println(a);
        Collections.reverse(a);
        System.out.println("reverse: " + a);
        Collections.sort(a);
        System.out.println("sort: " + a);
        Collections.reverse(a);
        System.out.println("descending: " + a);
    }
}
```

```
<terminated> SortElements (1) [Java Application]
[14, 3, 14, 2, 68, 12]
reverse: [12, 68, 2, 14, 3, 14]
sort: [2, 3, 12, 14, 14, 68]
descending: [68, 14, 14, 12, 3, 2]
```

SORT ELEMENTS USING COMPARABLE AND COMPARATOR:

To sort elements of non-primitive datatype, that is custom ClassName types, we have to over-ride methods of Comparable and Comparator interfaces as follows:

- **compareTo(Object o) of Comparable interface**
- **compare(Object o1, Object o2) of Comparator interface**

Steps to over-ride compareTo(Object o) of Comparable interface:

1. Make the class as a child to Comparable interface, using implements keyword
2. compareTo(Object o) should compare the properties of the current object (self) with the passed object using down-casted reference.
3. The method should return an integer as follows:
    - 0 if both states are same
    - +ve integer if current object state is higher
    - -ve integer if current object state is lower

Example program for Comparable interface:

Sample.java | SortObjects... | Example2.java | Example3.java | Example1.java

```
1  package collectionInterface;
2  import java.util.ArrayList;
4  class Student8 implements Comparable<Student8>{
5      int sid;
6      String name;
7      Student8(int sid, String name){
8          this.sid = sid;
9          this.name = name;
10     }
11     public String toString() {
12         return "sid=" + sid + " name=" + name;
13     }
14 //  public int compareTo(Student8 s) {
15 //      return this.name.compareTo(s.name);
16 //  }
17     public int compareTo(Student8 s) {
18         return this.sid - s.sid;
19     }
20 }
21 public class SortObjectsComparable {
22     public static void main(String[] args) {
23         ArrayList<Student8> al = new ArrayList<Student8>();
24         al.add(new Student8(101, "Tom"));
25         al.add(new Student8(102, "Jerry"));
26         al.add(new Student8(105, "Spike"));
27         al.add(new Student8(103, "Popeye"));
28         al.add(new Student8(104, "Bluto"));
29         System.out.println("************BEFORE SORTING************");
30         for(Student8 ele : al) {
31             System.out.println(ele);
32         }
33         Collections.sort(al);
34         System.out.println("************AFTER SORTING************");
35         for(Student8 ele : al) {
36             System.out.println(ele);
37         }
38         Collections.reverse(al);
39         System.out.println("************AFTER REVERSING************");
40         for(Student8 ele : al) {
41             System.out.println(ele);
42         }
43     }
44 }
```

Declaration | Console

```
<terminated> SortObjectsComparable [Java Applicatic
************BEFORE SORTING************
sid=101 name=Tom
sid=102 name=Jerry
sid=105 name=Spike
sid=103 name=Popeye
sid=104 name=Bluto
************AFTER SORTING************
sid=101 name=Tom
sid=102 name=Jerry
sid=103 name=Popeye
sid=104 name=Bluto
sid=105 name=Spike
************AFTER REVERSING************
sid=105 name=Spike
sid=104 name=Bluto
sid=103 name=Popeye
sid=102 name=Jerry
sid=101 name=Tom
```

Steps to over-ride compare(Object o1, Object o2) of Comparator interface:

1. A new implementing class for each attribute can be created to implement Comparator interface.
2. compare(Object o1, Object o2) should compare the properties of first object with the second object using two down-casted references.
3. The method should return an integer as follows:
    - ❖ 0 if both states are same
    - ❖ +ve integer if current object state is higher
    - ❖ -ve integer if current object state is lower

Example program for Comparator interface:

```
class SortObjectsComparator {
    public static void main(String[] args) {
        ArrayList<Student3> al = new ArrayList<Student3>();
        al.add(new Student3(101, "Tom"));
        al.add(new Student3(102, "Jerry"));
        al.add(new Student3(105, "Spike"));
        al.add(new Student3(103, "Popeye"));
        al.add(new Student3(104, "Bluto"));
        System.out.println("***********BEFORE SORTING***********");
        for(Student3 ele : al) {
            System.out.println(ele);
        }
        Collections.sort(al, new IdComparator());
        System.out.println("***********AFTER ID SORTING***********");
        for(Student3 ele : al) {
            System.out.println(ele);
        }
        Collections.sort(al, new NameComparator());
        System.out.println("***********AFTER NAME SORTING***********");
        for(Student3 ele : al) {
            System.out.println(ele);
        }

    }
}
```

```
package collectionInterface;
import java.util.ArrayList;
class Student3{
    int sid;
    String name;
    Student3(int sid, String name){
        this.sid = sid;
        this.name = name;
    }
    public String toString() {
        return "sid=" + sid + " name=" + name;
    }
}

class IdComparator implements Comparator<Student3>{
    public int compare(Student3 s1, Student3 s2) {
        return s1.sid - s2.sid;
    }
}

class NameComparator implements Comparator<Student3>{
    public int compare(Student3 s1, Student3 s2) {
        return s1.name.compareTo(s2.name);
    }
}
```

```
<terminated> SortObjectsComparator [Java Application] C:\Use
***********BEFORE SORTING***********
sid=101 name=Tom
sid=102 name=Jerry
sid=105 name=Spike
sid=103 name=Popeye
sid=104 name=Bluto
***********AFTER ID SORTING***********
sid=101 name=Tom
sid=102 name=Jerry
sid=103 name=Popeye
sid=104 name=Bluto
sid=105 name=Spike
***********AFTER NAME SORTING***********
sid=104 name=Bluto
sid=102 name=Jerry
sid=103 name=Popeye
sid=105 name=Spike
sid=101 name=Tom
```

Thus, the five operations with elements or objects of ArrayList are covered.

SET INTERFACE

SET INTERFACE

1. in-built interface
2. java.util package
3. not maintained
4. absent
5. not allowed
6. only one null
7. no constructor

characteristics

1. WHAT?
2. WHERE?
3. INSERTION ORDER
4. INDEXING
5. DUPLICATES
6. NULL
7. CONSTRUCTORS

HASHSET CLASS

1. concrete class
2. java.util package
3. not maintained
4. absent
5. not allowed
6. only one
7. HashSet()
   HashSet(Collection c)

LINKEDHASHSET CLASS

1. concrete class
2. java.util package
3. is maintained
4. absent
5. not allowed
6. only one
7. LinkedHashSet()
   LinkedHashSet(Collection c)

TREESET CLASS

1. concrete class
2. java.util package
3. it is of Comparable type, sorting happens internally, insertion order is maintained
4. absent
5. not allowed
6. null not allowed, only homogenous data
7. TreeSet()
   TreeSet(Collection c)

Program implementation for TreeSet class:

```java
class TreeSetEg {
    public static void main(String[] args) {
        // ADD ELEMENTS
    TreeSet<String> ts = new TreeSet<String>(Arrays.asList("tom","jerry","popeye","bluto"));
        System.out.println(ts);
    TreeSet<String> tr = new TreeSet<String>();
        tr.add("spike");
        tr.add("olive");
        System.out.println(tr);
        ts.addAll(tr);
        System.out.println(ts);
        // ACCESS ELEMENTS
        System.out.println("*****FOR EACH LOOP*****");
        for(String ele : ts) {
            System.out.println(ele);
        }
        System.out.println("****ITERATOR METHOD*****");
        Iterator<String> i = ts.iterator();
        while(i.hasNext()) {
            System.out.println(i.next());
        }
        System.out.println("following does not work for set interface: ");
        System.out.println("LIST ITERATOR() AND LIST ITERATOR(INT INDEX)");
        System.out.println("GET(INT INDEX) METHOD");
        // SEARCH ELEMENTS
        System.out.println(ts.contains("tom"));
        System.out.println(ts.contains("leo"));
        System.out.println(ts.containsAll(tr));
        // REMOVE ELEMENTS
        System.out.println(ts);
        ts.remove("jerry");
        System.out.println(ts);
        ts.removeAll(tr);
        System.out.println(ts);
        ts.addAll(tr);
        System.out.println(ts);
        ts.retainAll(tr);
        System.out.println(ts);
        System.out.println("sorting is not necessary for treeset as it is already sorted");
        TreeSet<String> re = (TreeSet<String>) ts.descendingSet();
        System.out.println("reverse the tree set: " + re);
    }
}
```

```
<terminated> TreeSetEg (1) [Java Application] C:\Users\naray\.p2\pool\plugins\org.ecl
[bluto, jerry, popeye, tom]
[olive, spike]
[bluto, jerry, olive, popeye, spike, tom]
*****FOR EACH LOOP*****
bluto
jerry
olive
popeye
spike
tom
****ITERATOR METHOD*****
bluto
jerry
olive
popeye
spike
tom
following does not work for set interface:
LIST ITERATOR() AND LIST ITERATOR(INT INDEX)
GET(INT INDEX) METHOD
true
false
true
[bluto, jerry, olive, popeye, spike, tom]
[bluto, olive, popeye, spike, tom]
[bluto, popeye, tom]
[bluto, olive, popeye, spike, tom]
[olive, spike]
sorting is not necessary for treeset as it is already sorted
reverse the tree set: [spike, olive]
```

MAP INTERFACE

Program Implementation for TreeMap class:

```java
import java.util.Map;

class TreeMapEg {
    public static void main(String[] args) {
        TreeMap<Integer, String> tm = new TreeMap<Integer, String>();
        // ADD ELEMENTS
        tm.put(101, "baahubali"); tm.put(102, "kumaravarma"); tm.put(103, "sivagami"); tm.put(104, "devasena");
        System.out.println(tm);
        TreeMap<Integer, String> tr = new TreeMap<Integer, String>();
        tr.put(201, "kattapa"); tr.put(202, "pingala");
        System.out.println(tr);
        tm.putAll(tr);
        System.out.println(tm);
        // ACCESS ELEMENTS
        System.out.println("FETCH VALUE FROM KEY: " + tm.get(102));
        System.out.println("PRINT KEYS AS LIST: " + tm.keySet());
        System.out.println("PRINT VALUES AS LIST: " + tm.values());
        System.out.println("CONVERT MAP TO LIST: " + tm.entrySet());
```

Console

```
<terminated> TreeMapEg [Java Application] C:\Users\naray\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.6.v20250130-0529\jre\bin\j
{101=baahubali, 102=kumaravarma, 103=sivagami, 104=devasena}
{201=kattapa, 202=pingala}
{101=baahubali, 102=kumaravarma, 103=sivagami, 104=devasena, 201=kattapa, 202=pingala}
FETCH VALUE FROM KEY: kumaravarma
PRINT KEYS AS LIST: [101, 102, 103, 104, 201, 202]
PRINT VALUES AS LIST: [baahubali, kumaravarma, sivagami, devasena, kattapa, pingala]
CONVERT MAP TO LIST: [101=baahubali, 102=kumaravarma, 103=sivagami, 104=devasena, 201=kattapa, 202=pingala]
****FETCH KEYS****
```

```
        System.out.println("CONVERT MAP TO LIST:   + tm.entrySet());
        System.out.println("****FETCH KEYS****");
        for(Integer key : tm.keySet()) {
            System.out.println(key);
        }
        System.out.println("****FETCH VALUES****");
        for(String value : tm.values()) {
            System.out.println(value);
        }
        System.out.println("****FETCH KEYS AND VALUES****");
        for(Map.Entry<Integer, String> ele : tm.entrySet()) {
            System.out.println("get value using get(Object key) method: " + tm.get(ele.getKey()));
            System.out.println("Key: " + ele.getKey() + " Value: " + ele.getValue());
        }
        // SEARCH ELEMENTS
        System.out.println("KEY SEARCH: " + tm.containsKey(201));
        System.out.println("VALUE SEARCH: " + tm.containsValue("devasena"));
        int targetKey = 103;
        String targetValue = "sivagami";
        for(Map.Entry<Integer, String> ele : tm.entrySet()) {
            if((ele.getKey() == targetKey) && (ele.getValue().equals(targetValue))){
                System.out.println("entry found " + ele.getKey() + " " + ele.getValue());
            }
        }
        // REMOVE ELEMENTS
        System.out.println("remove 101: " + tm.remove(101));
        System.out.println("remove 202 pingala: " + tm.remove(202, "pingala"));
        System.out.println(tm);
        System.out.println(tm.size());
        tm.clear();
        System.out.println(tm);
        System.out.println(tm.size());
        System.out.println(tm.isEmpty());
    }
}
```

```
CONVERT MAP TO LIST: [101=baahubali, 102=kumaravarma, 103=s
****FETCH KEYS****
101
102
103
104
201
202
****FETCH VALUES****
baahubali
kumaravarma
sivagami
devasena
kattapa
pingala
****FETCH KEYS AND VALUES****
get value using get(Object key) method: baahubali
Key: 101 Value: baahubali
get value using get(Object key) method: kumaravarma
Key: 102 Value: kumaravarma
get value using get(Object key) method: sivagami
Key: 103 Value: sivagami
get value using get(Object key) method: devasena
Key: 104 Value: devasena
get value using get(Object key) method: kattapa
Key: 201 Value: kattapa
get value using get(Object key) method: pingala
Key: 202 Value: pingala
KEY SEARCH: true
VALUE SEARCH: true
entry found 103 sivagami
remove 101: baahubali
remove 202 pingala: true
{102=kumaravarma, 103=sivagami, 104=devasena, 201=kattapa}
4
{}
0
true
```

These are program examples for Set and Map interfaces.